Crazy Mototaxi (Nombre WIP)

Avance de Proyecto de Computación Gráfica · Semana 10

1. ¿Dónde estamos?

En la semana 2 propusimos un juego de conducción en mototaxi sobre un mapa 3D generado proceduralmente e inspirado en la sierra y la selva peruanas. Ocho semanas después, la base técnica ya funciona: el código pesado en C++ se compila como extensión nativa de Godot, el terreno se genera a partir de una semilla y sobre él se trazan caminos.

En el camino el proyecto cambió de meta. El profesor nos indicó que el terreno debe implementar el paper **Terrain Amplification using Multi-scale Erosion** (Schott et al., ACM TOG 2024), y que esa es la valla a superar. Además, el mapa ahora incluye la costa, para cubrir las tres regiones del Perú y sus transiciones.

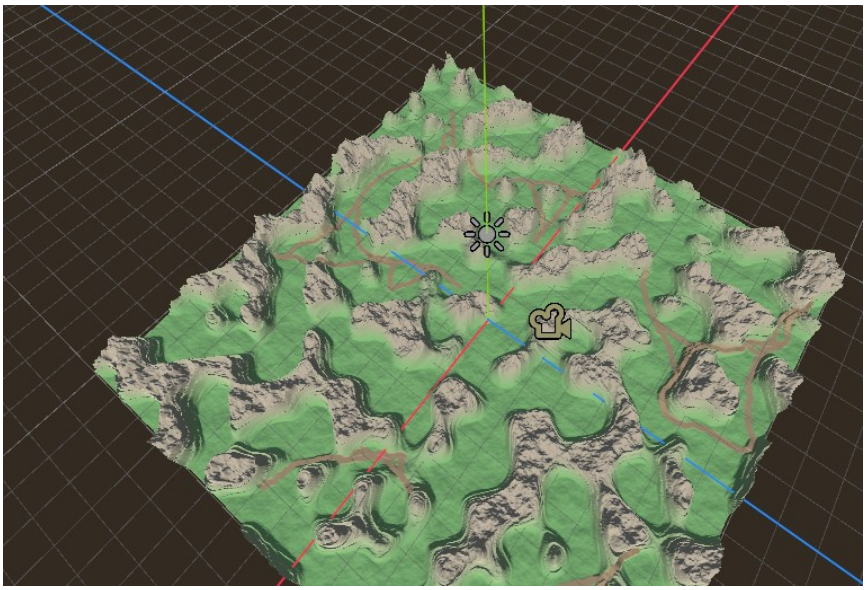


Figura 1. Estado actual en el editor de Godot: terreno 128 × 128 con ruido Perlin, dos biomas y caminos A.*

Lo que ya está hecho

- **Integración C++ y Godot** mediante GDExtension: la generación se compila como .dll y se llama desde GDScript, también dentro del editor.
- **Ruido Perlin propio** con fBm de 6 octavas y dos mapas de control (elevación y humedad) que mezclan sierra y selva con una banda de transición suave.
- **Red de caminos:** puntos de interés separados entre sí, unidos con sus k vecinos más cercanos y trazados con A* sobre la grilla de alturas. El terreno se aplana alrededor de cada camino.
- **Malla propia:** vértices, índices, normales por diferencias centrales y color por vértice según bioma.
- **Reproducción del paper** desde su repositorio oficial (H-Schott/MultiScaleErosion) y lectura del 75% del artículo.

2. Lo que aprendimos

- **El profesor fijó el paper como meta.** El terreno final debe salir de la erosión multiescala; el ruido solo ya no basta.
- **El ruido no tiene agua.** En la Figura 1 no hay valles que drenen ni ríos continuos. La erosión fluvial agrega justamente eso, porque esculpe el terreno siguiendo el camino del agua.
- **Nuestro generador no se descarta.** Se convierte en el terreno de entrada T0 que el paper amplifica. El propio artículo muestra que su método funciona sobre terrenos hechos con ruido procedural.
- **No necesitamos GPU.** El paper corre cada paso de erosión en menos de 11 ms en GPU, pero nosotros generamos el mapa una sola vez por partida. Una versión en C++ sobre CPU, durante una pantalla de carga, alcanza y nos ahorra portar compute shaders.
- **La dureza conecta erosión y biomas.** El paper controla la erosión con un mapa de dureza $\rho(p)$ y una pendiente crítica $s_0(p)$. Si cada región tiene sus propios valores, la costa, la sierra y la selva se erosionan distinto de forma natural.
- **Un solo mesh no escala.** A 1024 × 1024 el mapa necesita dividirse en chunks y usar colisión por heightmap (HeightMapShape3D).

3. Meta final: Computación Gráfica

El objetivo es la generación autónoma de un terreno que se vea moldeado por el agua, con costa, sierra y selva y transiciones entre ellas. A partir de una semilla, el pipeline hace lo siguiente:

1. **Mapa base T0 (256 × 256):** nuestro ruido más un gradiente oeste a este (costa, sierra, selva), deformado con ruido para que las fronteras no sean rectas.
2. **Amplificación multiescala (256 → 512 → 1024):** en cada nivel se sube la resolución con interpolación bicúbica y se aplican tres pasos, en este orden.
    - **Erosión fluvial:** $h \leftarrow h - k\,(1-\rho)\cdot \min(s^n, s_{max}^n)\cdot \min(a^m, a_{max}^m)$, donde s es la pendiente y a el área de drenaje (cuánta agua pasa por cada celda), calculada con flow routing ($p = 1.3$).
    - **Estabilización térmica:** cada celda sube o baja según cuántos vecinos superan la pendiente crítica s_0, lo que suaviza los cortes demasiado verticales.
    - **Deposición:** el material erosionado viaja con el agua y se deposita en los valles, formando fondos planos.
3. **Retargeting por difusión:** recupera la altura de picos y crestas que la erosión rebajó.
4. **Breaching multiescala (opcional):** elimina pozos para que toda el agua drene hacia los bordes.
5. **Malla por chunks y shader propio:** texturas según bioma, pendiente y altura; los ríos se pintan directamente desde el mapa de drenaje a, que el pipeline ya calcula.

Región	Dureza ρ	Pendiente crítica s_0	Resultado esperado
Costa	Baja	Baja	Llanuras y quebradas secas, con mucha deposición.
Sierra	Alta, con ruido fractal	Alta	Crestas, cañones y laderas empinadas y variadas.

Región	Dureza ρ	Pendiente crítica s_0	Resultado esperado
Selva	Media	Media	Colinas suaves y ríos anchos con sedimento.

Valores iniciales, a calibrar en la semana 11 comparando contra la reproducción del paper.

4. Meta final: Videojuego

Una mototaxi cruza el terreno como loca. Recoge pasajeros y debe llevarlos a su destino lo antes posible, por la ruta recomendada o por el atajo que el jugador elija. Si tarda, el pasajero se enoja y la propina baja. La mototaxi tiene vida: los choques fuertes y los vuelcos la dañan, y si llega a cero termina la partida.

El ciclo de juego es: explorar → encontrar pasajero → recogerlo → recibir destino y tiempo → elegir ruta → entregar o fallar. Los puntos de recogida y destino salen del propio terreno: picos (drenaje bajo), confluencias de ríos (drenaje alto) y llanuras de la costa.

5. El 20% que da el 80%

Criterio de prioridad: una actividad entra al plan solo si se ve en la demo final o si el profesor la evalúa directamente. Estas son las siete que sostienen el proyecto:

1. **Erosión multiescala en C++** (fluvial, térmica y deposición). Es la valla del curso.
2. **Costa, sierra y selva conectadas a la erosión** mediante la dureza ρ y la pendiente s_0.
3. **Shader de terreno** por bioma, pendiente y ríos. Es lo primero que se ve en la demo.
4. **Chunks y colisión** para que el mapa de 1024 × 1024 sea jugable.
5. **Mototaxi con la física de Godot** (VehicleBody3D) que se sienta bien en pendientes.
6. **Ciclo de carrera completo:** pasajero, destino, tiempo, vida y fin de partida.
7. **Ruta recomendada con A*** sobre el terreno ya erosionado, reutilizando el código de caminos.

Se recorta o queda para el final: LOD, generación en segundo plano, mundo infinito, port a GPU, daño detallado, personalización, eventos especiales e IA de pasajeros.

Contrato entre las dos áreas

Para que nadie espere a nadie, en la semana 11 se acuerda una interfaz única, TerrainWorld. CG la implementa de verdad; el área de juego trabaja desde el día 1 con una versión provisional (stub) sobre el terreno actual.

Función	Para qué la usa el juego
generate(seed)	Construir el mapa completo al iniciar la partida.
get_height(x, z), get_normal(x, z)	Colocar objetos y pasajeros sobre el suelo.
get_biome(x, z)	Pesos costa, sierra y selva (sonido, ambiente, puntaje).
get_drainage(x, z)	Saber dónde hay ríos.
get_landmarks()	Lista de puntos de recogida y destino con su tipo.
find_route(a, b)	Ruta recomendada como lista de puntos.

6. Cronograma: semanas 11 a 15

Roles: **CG-A** (erosión), **CG-B** (mundo: biomas, chunks, shader), **J-A** (vehículo y vida), **J-B** (carreras, rutas e interfaz). Supuesto de carga: unas 15 horas por persona por semana (3 horas diarias de lunes a viernes), es decir, unas 300 horas de equipo en total. Las horas de cada celda son estimaciones con IA como apoyo y ya incluyen depuración.

Sem.	CG-A · Erosión	CG-B · Mundo	J-A · Vehículo	J-B · Carreras
S11	Heightfield en C++, flow routing, erosión fluvial con límites y térmica a una sola resolución. Validar contra la reproducción oficial con el mismo input. (~15 h)	T0 con gradiente costa, sierra y selva; mapas ρ y s_0 por región; publicar la API TerrainWorld con stub. (~12 h)	Mototaxi con VehicleBody3D, cámara de persecución y controles, probada en el terreno actual. (~15 h)	Estados de carrera (libre, con pasajero, entregado, fallado), zonas de recogida con Area3D, temporizador y HUD básico. (~15 h)
S12	Bucle multiescala 256 → 512 → 1024 con upsampling bicúbico y deposición. Medir tiempos; si supera ~30 s, paralelizar filas con std::thread. (~18 h)	Chunks de 128 × 128 sobre el heightmap de 1024 y colisión con HeightMapShape3D; shader de splatting con texturas libres (CC0). (~18 h)	Vida: daño por impulso de choque, detección de vuelco, fin de partida y reinicio. (~12 h)	Ruta A* dibujada en el mundo y flecha de navegación; tiempo límite según largo y pendiente de la ruta. (~15 h)
S13	Retargeting por difusión; semilla → mapa completo con pantalla de carga. Breaching si va a tiempo. (~15 h)	Ríos desde el drenaje en el shader; landmarks automáticos; caminos regenerados sobre el terreno erosionado. (~15 h)	Integrar con la API real; ajustar suspensión y agarre en el terreno erosionado. (~15 h)	Integrar con la API real; pasajero que se enoja, propina y puntaje. (~15 h)
S14	Figuras de cada etapa del pipeline y tabla de tiempos por resolución para la presentación. (~10 h)	Pulido visual: transiciones, niebla, cielo y agua. Vegetación con MultiMesh solo si sobra tiempo. (~12 h)	Sonido de motor y choques; balance de vida y daño. (~10 h)	Menú con semilla, minimapa a partir del heightmap y pantalla final con puntaje. (~12 h)
S15	Todo el equipo: pruebas con 3 a 5 compañeros, corrección de bugs, video demo, diapositivas y README. Sin funciones nuevas.			

Sem.	Hito verificable al cierre de la semana
S11	Imagen antes y después de la erosión a 256 × 256. Mototaxi manejable que recoge a un pasajero.
S12	Mapa de 1024 × 1024 erosionado y texturizado que se puede recorrer. Partida completa que termina por tiempo o por vida.
S13	Juego completo con una semilla nueva en cada partida. Es la semana de mayor riesgo: si algo se atrasa, se recorta de S14, nunca de S13.
S14	Versión candidata que alguien externo puede jugar sin explicación. Congelamiento de funciones al terminar la semana.
S15	Entrega final.

7. Requerimientos actualizados

Nivel	Requerimiento	Estado	Comentario
Must Have	Terreno procedural con semilla	Hecho	Pasa a ser el T0 del paper.
Must Have	Erosión multiescala del paper	Pendiente	Nuevo. S11 y S12.
Must Have	Costa, sierra y selva con transiciones	En curso	Falta la costa. S11.
Must Have	Construcción propia de mallas	Hecho	Se adapta a chunks en S12.
Must Have	Gestión por chunks	Pendiente	S12.
Must Have	Mototaxi controlable	Pendiente	S11.
Must Have	Pasajeros y carreras	Pendiente	S11 a S13.
Must Have	Landmarks básicos	Pendiente	Salen del drenaje. S13.
Must Have	Vida y fin de partida	Pendiente	S12.
Should Have	Rutas recomendadas	En curso	A* hecho; falta usarlo en el juego.
Should Have	Shaders propios por bioma	Pendiente	S12.
Should Have	Tiempo según dificultad	Pendiente	S12.
Should Have	Retargeting y ríos	Pendiente	S13.
Nice to Have	Breaching multiescala	Opcional	Solo si S13 va a tiempo.
Nice to Have	Minimapa	Opcional	S14.
Nice to Have	Generación multihilo	Opcional	Solo si la generación es lenta.
Recortado	LOD, daño detallado, personalización, eventos	Fuera	No cambian la nota ni la demo.

8. Riesgos y cómo los cubrimos

- **La erosión en CPU tarda demasiado.** Generar como máximo a 1024 × 1024, usar menos iteraciones en las resoluciones altas (como en la Tabla 1 del paper) y paralelizar por filas.
- **La mototaxi se atasca o vuelca en el terreno erosionado.** Los caminos aplanan el suelo, la estabilización térmica suaviza pendientes y la suspensión se ajusta en S13.
- **La integración llega tarde.** API acordada en S11, stub desde el primer día y fusión a la rama principal cada viernes con una demo de 5 minutos.
- **Código generado con IA que nadie entiende.** Regla del equipo: quien sube una ecuación del paper la explica a los demás, y cada paso se valida con una imagen contra la reproducción oficial.

Fuera del alcance

Port de la erosión a GPU, motor de físicas propio, mundo infinito, LOD, IA compleja para pasajeros y una red vial completamente realista.